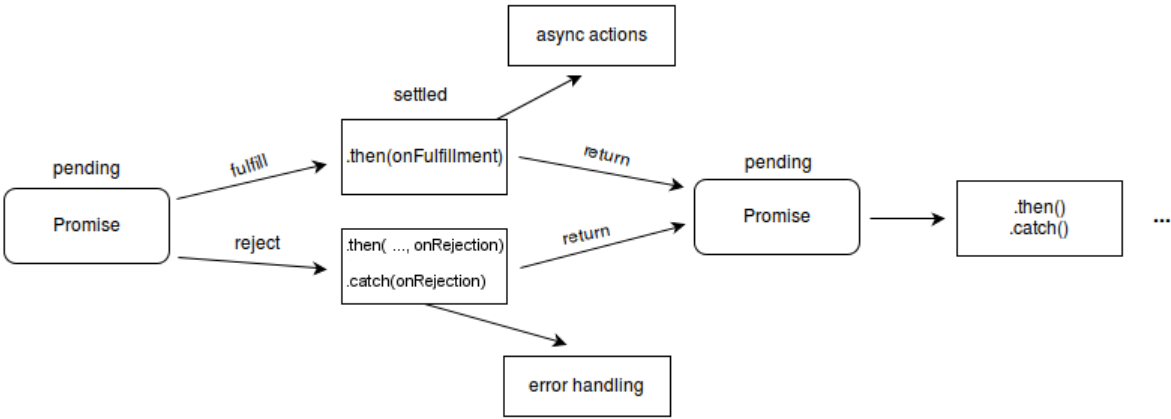


2. Promise

프로미스(Promise)는 비동기 처리를 추상화한 객체로 비동기 처리를 실행하고 그 처리가 끝난 후에 다음 처리를 실행하기 위한 용도로 사용됩니다.

Promise 객체는 다음 중 하나의 상태로 존재합니다.

- Fulfilled : 작업이 성공한 경우
- Rejected : 작업이 실패한 경우. 에러를 반환한다.
- Pending : 작업이 아직 진행 중인 경우



프로미스 처리 흐름 - 출처 : MDN

참고 : [mdn Promise](#)

비동기 처리 방식으로 실행된 결과를 성공과 실패를 관리하는 객체로 생성하는 과정과 최종 결과로 나뉘어 집니다.

1. Promise 생성 과정

- 대기(pending)
 - 요청한 실행의 반환을 기다리는 상태 (코드 작성 없음 , 자동)
- 성공(resolved)
 - 요청한 실행이 성공한 상태 (코드 작성)
- 실패(rejected)
 - 요청한 실행이 실패한 상태 (코드 작성)

2. Promise 최종 결과

- 최종 성공 (fulfilled)
 - 성공적으로 원하는 결과값을 반환하는 경우
- 최종 실패 (unfulfilled)
 - 요청이 실패해서 오류 처리를 해야 하는 경우

1. Promise 생성자

```
var promise = new Promise(function(resolve, reject) { ... });
```

- `resolve(obj)` : 처리가 성공적으로 완료되었을 때는 `resolve` 함수를 호출.
 - `resolve` 함수는 프로미스를 종료.
 - `obj` : `resolve` 함수에 인자값을 전달
 - thenable이면 이행값 전달
- `reject(err)` : 처리가 실패되었다면 `reject` 함수를 호출
 - `reject` 함수는 프로미스를 종료
 - `err` : `reject` 함수에 인자값을 전달하는데 주로 오류 메시지 문자열을 인수로 사용

2. Promise API

- **Promise 인스턴스에서 사용 가능한 주요 함수**
 - `promise.then(onFulfilled, onRejected)` :
 - `onFulfilled` 함수 : 성공 콜백 함수라고 하며 프로미스 내부의 처리가 정상적으로 완료될 때 호출되는 함수로 프로미스 안에서 `resolve` 함수를 실행할 때 전달되는 인수로 `response`를 받는다.
 - `onRejected` 함수 : 실패 콜백 함수라고 하며 프로미스의 처리가 실패로 끝날 때 호출되며 프로미스 안에서 `reject` 함수를 실행할 때 전달되는 인수로 `error`를 받는다.
 - `promise.catch(onRejected)` : `promise.then(undefined, onRejected)`에 대한 편리한 버전
 - `onRejected` 함수 : 실패 콜백 함수라고 하며 프로미스의 처리가 실패로 끝날 때 호출되며 프로미스 안에서 `reject` 함수를 실행할 때 전달되는 인수로 `error`를 받는다.
 - `promise.finally(onFinally)` : 성공 또는 실패 여부와 상관없이 무조건 해야 하는 처리
 - `finally`는 프로미스가 성공(`fulfilled`) 또는 실패(`rejected`) 상태일 때 호출되는 메서드로 프로미스 체인의 가장 마지막에 사용됩니다.

```
{% code lineNumbers="true" %}
```

```
function accessCall(msg) {
    console.log("성공 : async call ", msg);
}

function failCall(msg) {
    console.log("실패 : reject call", msg);
}

var fPromise = function(param) {
    console.log("비동기 시작");
    return new Promise(function(resolve, reject){
        setTimeout(function() {
            if (param) {
                resolve("ok");
            } else {
                reject("fail");
            }
        }, 1000);
    });
}

console.log("동기 시작");
var rn = fPromise(true)
```

```

    .then(function(msg) {
        console.log("성공 : resolve ", msg);
        accessCall(msg);
    })
    .catch(function(msg) {
        console.log("실패 : reject ", msg);
        failCall(msg);
    })
    .finally(function() {
        console.log("비동기 종료 ");
    });
console.log("동기 종료 :: ", rn);

```

{% endcode %}

- 22 line : 동기 시작 출력
- 23 line : Promise 함수 fPromise 호출 1초 후 로직 수행
- 10 line : 비동기 시작 출력
- 35 line : 동기 종료 출력 Promise 는 pending 상태
- 13 line : param 이 true 이므로 resolve 함수 호출, 인자로 "ok" 전달
- 24 line : 성공 메시지 표시 ('성공 : resolve ok')
- 25 line : accessCall 함수 호출
- 2 line : 메시지 표시 ('성공 : async call ok')
- 32 line : 비동기 종료 메시지 표시

동기 시작

비동기 시작

동기 종료 :: ▶ *Promise {<pending>}*

✖ undefined

성공 : resolve ok

성공 : async call ok

비동기 종료

실패 호출 : 성공과 동일한 호출 구조

```

console.log("동기 시작");
var rn = fPromise(false)
    .then(function(msg) {
        console.log("성공 : resolve ", msg);
        accessCall(msg);
    })
    .catch(function(msg) {
        console.log("실패 : reject ", msg);
        failCall(msg);
    })
    .finally(function() {
        console.log("비동기 종료 ");
    });

```

```
});
console.log("동기 종료 :: ", rn)
```

동기 시작

비동기 시작

동기 종료 :: **▶ Promise {<pending>}**

↳ undefined

실패 : reject fail

실패 : reject call fail

비동기 종료

- 다음 코드를 확인 해 봅니다.

```
function delayCall(milliseconds, callBack) {
  console.log("delayCall ... Start .... Sync");
  var p1 = new Promise((resolve, reject) => {
    console.log("delayCall ... Start .... ASync");
    setTimeout(() => {
      console.log("delayCall ... Promise ....resolve(callBack(rn))")
    }, milliseconds);
    resolve(callBack(rn));
  });

  var rn = {
    "Date": new Date(),
    "callBack": callBack,
    "msg" : "ok"
  }

  p1.then((value) => {
    console.log(`delayCall resolve .. End .. ASync .. ${value} ....
    ${new Date().getSeconds()}`);
  }).catch((value)=>{
    console.log(`delayCall reject .. End .. ASync .. ${value} ....
    ${new Date().getSeconds()}`);
  })
  console.log("delayCall ... End .... Sync");
  return "OK";
}

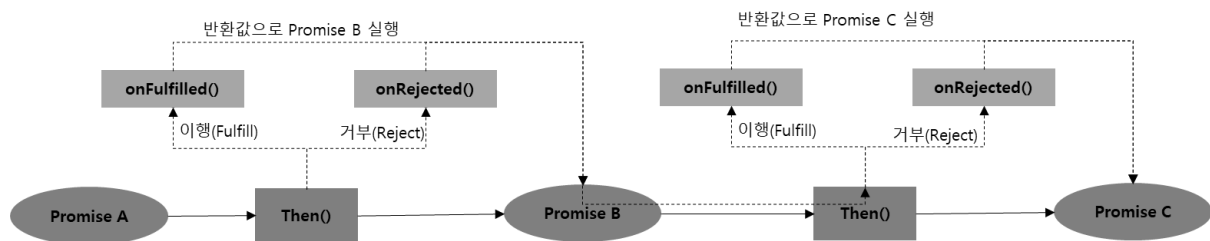
function fFunc01(param) {
  console.log(`fFunc .... ${new Date().getSeconds()} param ::\n`, param)
  ;
  return param.msg;
}

console.log(`##### Start .... ${new Date().getSeconds()}`);
var rnDelayCall = delayCall(1000, fFunc01);
```

```
console.log(`##### End .... ${new Date().getSeconds()} .... dcall :: ${rnDelayCall}`);
```

```
##### Start .... 22 VM143:28
delayCall ... Start .... Sync VM143:2
delayCall ... Start .... ASync VM143:4
delayCall ... End .... Sync VM143:21
##### End .... 22 .... dcall :: OK VM143:30
undefined
delayCall ... Promise ....resolve(callback(rn)) VM143:6
fFunc .... 23 param :: VM143:25
  ▶ {Date: Wed Dec 27 2023 22:16:23 GMT+0900 (한국 표준시), msg: 'ok', callback: f}
delayCall resolve .. End .. ASync .. ok .... 23 VM143:17
```

3. Promise 체인 실행 호출



사칙 연산을 예제로 Promise 체인 실행 호출을 확인 해 봅니다.

```
// 기명 함수
const sum = function sum (num1, num2) {
  console.log("sum : pending" );
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      let result = num1 + num2;
      result = `${num1} + ${num2} = ${result}`;
      resolve(result);
    }, 2000);
  });
}

// 익명 함수
const minus = function (num1, num2) {
  console.log("minus : pending" );
  return new Promise(function(resolve, reject) {
    setTimeout(function() {
      let result = num1 - num2;
      result = `${num1} - ${num2} = ${result}`;
      resolve(result);
    }, 1500);
  });
}
```

```
// 화살표 함수 적용
const multiply = (num1, num2) => {
  console.log("multiply : pending" );
  return new Promise((resolve, reject) => {
    setTimeout( () => {
      let result = num1 * num2;
      result = `${num1} * ${num2} = ${result}`;
      resolve(result);
    }, 1000);
  });
}

// 화살표 함수 적용
const divide = (num1, num2) => {
  console.log("divide : pending" );
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let result = num1 / num2;
      result = `${num1} / ${num2} = ${result}`;
      resolve(result);
    }, 1000);
  });
}
```

3-1. 비동기 순차 처리

```
sum(100, 200).then((response) => {
  console.log("sum resolve : " + response);
  return minus(100, 200);
}).then( (response) => {
  console.log("minus resolve : " + response);
  return multiply(100, 200);
}).then((response) => {
  console.log("multiply resolve : " + response);
  return divide(100, 200);
}).then((response) => {
  console.log("divide resolve : " + response);
});

console.log("비동식 처리!!");
```

```

console.log("비동식 처리!!");
sum : pending
비동식 처리!!
undefined
sum resolve : 100 + 200 = 300
minus : pending
minus resolve : 100 - 200 = -100
multiply : pending
multiply resolve : 100 * 200 = 20000
divide : pending
divide resolve : 100 / 200 = 0.5

```

3-1. Promise 정적 메서드

- `Promise.resolve(obj)` : 주어진 값으로 이행하는 `Promise.then` 객체를 반환
- `Promise.reject(reason)` : 주어진 이유(reason)로 거부된 `Promise` 객체를 반환

```

// Promise 정적 메서드 실행
Promise.resolve(sum(100, 200)).then((response) => {
    console.log("sum resolve : " + response);
});

```

3-2. 병렬처리 (Promise.all 메서드)

- `Promise.all` 메서드는 모든 처리가 성공적으로 끝났을 때만 다음 작업을 실행 합니다.
- `all` 메서드의 인수인 `iterable`은 `Promise` 객체가 요소로 들어 있는 반복 가능한(`Iterable`)한 객체입니다.
- 실행 시간이 서로 다른 사칙연산을 병렬로 모두 실행한 후에 그 결과값을 배열로 반환합니다.

```

Promise.all([
    sum(100, 200),
    minus(100, 200),
    multiply(100, 200),
    divide(100, 200)
]).then(function(response) {
    response.forEach( (item, index) => {
        console.log(`${index} :: ${item}` );
    })
});

```

```

sum : pending
minus : pending
multiply : pending
divide : pending
· ▶ Promise {<pending>}
0 :: 100 + 200 = 300
1 :: 100 - 200 = -100
2 :: 100 * 200 = 20000
3 :: 100 / 200 = 0.5

```

3-3. 병렬처리 (Promise.race 메서드)

- Promise.race 메서드는 가장 먼저 종료한 Promise 객체의 결과만 다음 작업으로 보냅니다.
- race 메서드의 인수인 iterable은 Promise 객체가 요소로 들어 있는 반복 가능한(Iterable)한 객체입니다.

```

Promise.race([
  sum(100, 200),
  minus(100, 200),
  multiply(100, 200),
  divide(100, 200)
]).then(function(response) {
  console.log(`${response}`);
});

```

```

sum : pending
minus : pending
multiply : pending
divide : pending
⏪ ▶ Promise {<pending>}
100 * 200 = 20000

```